

Plans d'exécution SQL

14 requêtes lentes



Lire un **EXPLAIN**, repérer le goulot, corriger, et savoir quel gain en attendre.

Une requête lente n'est presque jamais un mystère : le plan d'exécution dit déjà où ça bloque. Ce guide prend 14 requêtes qui rament en prod, lit leur plan, nomme le goulot, corrige, et donne l'ordre de grandeur du gain à en attendre. Dialecte : PostgreSQL, avec les écarts moteur signalés.

Observation Datafuji : la plupart des requêtes lentes que je vois passer ne sont pas mal écrites, elles sont juste illisibles pour l'optimiseur. Lance toujours **EXPLAIN (ANALYZE, BUFFERS)** avant de deviner. Schéma utilisé dans tout le guide : **clients, commandes, lignes_commande, produits**. Les gains affichés sont des ordres de grandeur typiques, pas des benchmarks mesurés sur un jeu de données précis : à vérifier sur ton propre volume.

Scans : quand la table entière défile.

Ici, le plan affiche un scan complet là où tu attendais un accès direct. La question n'est pas toujours "il manque un index", parfois l'index n'aidait même pas.

1. Un filtre quotidien qui balaie toute la table **INDEX MANQUANT**

Requête lente : Un dashboard interroge **commandes** (5 millions de lignes) par client, plusieurs fois par minute, sans aucun index sur la colonne de filtre.

```
SELECT * FROM commandes
WHERE client_id = 48210;
```

Lecture du plan : Seq Scan on commandes (cost=0.00..152340.00 rows=1) Filter: (client_id = 48210) Rows Removed by Filter: 4999968. Le coût est concentré dans le scan complet, pas dans le filtre lui-même.

Goulot : aucun index sur **client_id** : Postgres doit lire et tester chaque ligne de la table pour n'en garder qu'une poignée.

Correction : un index simple sur la colonne de filtre suffit ici, la sélectivité est excellente (un client sur des millions).

```
CREATE INDEX idx_commandes_client_id
ON commandes(client_id);
```

Gain typique : d'un scan complet de plusieurs secondes, le plan bascule sur un **Index Scan** de quelques lignes, de l'ordre de la seconde à quelques millisecondes.

2. Un index qui ne changerait rien **CONTRE-INTUITIF**

Requête lente : La même table **commandes** filtrée cette fois sur **statut**, une colonne qui ne prend que trois valeurs à peu près équiréparties.

```
SELECT * FROM commandes
WHERE statut = 'annulee';
```

Lecture du plan : Seq Scan on commandes rows=1650312 Filter: (statut = 'annulee'). Même avec un index posé sur **statut**, le planificateur choisirait quand même le Seq Scan.

Goulot : ce n'est pas un index manquant, c'est une faible sélectivité : un tiers des lignes matche, un accès par index

coûterait plus cher en lectures aléatoires qu'un scan séquentiel.

Correction : n'indexe pas **statut** seule. Si l'usage réel combine toujours "annulée" et une période récente, un index partiel cible juste ce sous-ensemble.

```
CREATE INDEX idx_commandes_annulees_recentes
ON commandes(date_commande)
WHERE statut = 'annulee';
```

Gain typique : sur la requête combinée (annulée + période récente), on passe d'un scan complet d'environ 1 seconde à un parcours d'index partiel de quelques millisecondes.

À retenir avant de poser un index

Un index n'accélère pas toujours une requête, il faut d'abord regarder la sélectivité du filtre.

Filtre sélectif

Peu de lignes matchent : l'index gagne, l'accès direct bat le scan complet.

Filtre peu sélectif

Beaucoup de lignes matchent : le scan complet peut rester le meilleur plan.

Jointures : boucle, mémoire ou ordre.

Ici, le plan choisit entre trois familles de jointure. Chacune a son propre goulot : une boucle non indexée, une table de hachage qui déborde, ou un filtre poussé trop tard.

3. Une boucle qui rescane la même table NESTED LOOP

Requête lente : On cherche les commandes des 5 000 clients au segment "premium" dans une table **commandes** de 5 millions de lignes sans index sur la clé de jointure.

```
SELECT c.nom, o.montant
FROM clients c
JOIN commandes o ON o.client_id = c.id
WHERE c.segment = 'premium';
```

Lecture du plan : Nested Loop -> Seq Scan on clients (rows=5000) -> Seq Scan on commandes (client_id = c.id). Le scan interne se répète pour chacune des 5 000 lignes externes.

Goulot : sans index sur **commandes.client_id**, chaque itération de la boucle relit une bonne partie de la table interne : le coût grimpe avec le produit des deux volumes, pas leur somme.

Correction : un index sur la clé de jointure interne transforme chaque itération en accès direct, ou pousse le planificateur vers un Hash Join.

```
CREATE INDEX idx_commandes_client_id
ON commandes(client_id);
```

Gain typique : une boucle de plusieurs secondes à 5 000 scans internes tombe à quelques centaines de millisecondes une fois la boucle indexée.

4. Une jointure qui déborde sur disque HASH JOIN

Requête lente : Un rapprochement massif entre **commandes** et **lignes_commande** juste après un gros import, avant tout **ANALYZE**.

```
SELECT o.id, l.produit_id, l.quantite
FROM commandes o
JOIN lignes_commande l ON l.commande_id = o.id;
```

Lecture du plan : Hash Join Buckets: 131072 Batches: 32 Memory Usage: 4096kB. Le nombre de lots ("Batches") supérieur à 1 indique que la table de hachage a débordé de **work_mem** et s'écrit sur disque.

Goulot : des statistiques obsolètes après l'import font sous-estimer le nombre de lignes, le planificateur dimensionne mal la table de hachage, qui déborde et multiplie les relectures disque.

Correction : rafraîchir les statistiques en premier réflexe, avant d'envisager de monter **work_mem** pour cette requête précise.

```
ANALYZE commandes;
ANALYZE lignes_commande;
```

Gain typique : en repassant à un seul lot bien dimensionné, une jointure de plusieurs secondes revient à moins de deux secondes; augmenter **work_mem** sans corriger les stats ne fait souvent que déplacer le problème.

5. Une CTE qui bloque le filtre en amont ORDRE DE JOINTURE, POSTGRESQL < 12

Requête lente : On ne veut que les commandes des clients actifs, mais la requête matérialise d'abord toute la table **commandes** dans une CTE.

```
WITH toutes_commandes AS (
  SELECT * FROM commandes
)
SELECT tc.*, c.nom
FROM toutes_commandes tc
JOIN clients c ON c.id = tc.client_id
WHERE c.statut = 'actif';
```

Lecture du plan : CTE Scan on toutes_commandes, précédé d'un scan complet de **commandes** sans aucun filtre poussé dessus, alors que seule une fraction des clients est active.

Goulot : sur PostgreSQL avant la version 12, une CTE est une barrière d'optimisation : elle est matérialisée en entier avant que le reste de la requête ne s'applique, le filtre **statut = 'actif'** arrive trop tard pour réduire le volume lu.

Correction : sur PostgreSQL 12 et plus, les CTE sans effet de bord sont automatiquement "inlinées"; sur des versions antérieures, remplace la CTE par une sous-requête classique ou force l'inlining explicitement.

```
WITH toutes_commandes AS NOT MATERIALIZED (
  SELECT * FROM commandes
)
SELECT tc.*, c.nom
FROM toutes_commandes tc
JOIN clients c ON c.id = tc.client_id
WHERE c.statut = 'actif';
```

Gain typique : en laissant le filtre client actif remonter avant la lecture de **commandes**, on passe d'un ordre de plusieurs secondes à moins d'une seconde; le comportement exact dépend de la version du moteur.

Index : présent, mais mal exploité.

Ici, un index existe déjà. Le problème n'est pas son absence, c'est que la requête ou l'index lui-même l'empêchent de servir pleinement.

6. SELECT * qui empêche l'Index Only Scan HEAP FETCHES

Requête lente : Un dashboard interroge très souvent les commandes d'un client sur une période récente; un index composite existe déjà sur (**client_id, date_commande**).

```
SELECT * FROM commandes
WHERE client_id = 48210
AND date_commande > '2026-01-01';
```

Lecture du plan : Index Scan using idx_commandes_client_date (rows=12) Heap Fetches: 12. Chaque ligne trouvée dans l'index déclenche quand même un aller-retour vers la table.

Goulot : **SELECT *** réclame des colonnes absentes de l'index (**montant, statut**), donc Postgres ne peut jamais faire un Index Only Scan : chaque ligne coûte un accès disque aléatoire en plus.

Correction : sélectionne uniquement les colonnes utiles et étend l'index en index couvrant avec **INCLUDE**, pour que tout soit lisible depuis l'index seul.

```
CREATE INDEX idx_commandes_client_date_cov
ON commandes(client_id, date_commande)
INCLUDE (montant, statut);
```

Gain typique : sur une requête répétée à haute fréquence, on passe d'environ 40 ms à quelques millisecondes une fois l'Index Only Scan actif (table régulièrement passée au **VACUUM**).

7. Un index gonflé par des lignes mortes INDEX PARTIEL

Requête lente : 40% des lignes de **commandes** sont des suppressions logiques (**deleted_at** renseigné), jamais interrogées en pratique, mais toujours indexées.

```
SELECT * FROM commandes
WHERE client_id = 48210
AND deleted_at IS NULL;
```

Lecture du plan : Index Scan using idx_commandes_client_id, Buffers: shared hit=210. L'index est utilisé, mais il est large : il référence aussi les lignes supprimées, jamais lues.

Goulot : l'index complet indexe des millions de lignes mortes pour rien, ce qui gonfle sa taille sur disque, ralentit son parcours et alourdit sa maintenance à chaque écriture.

Correction : un index partiel qui ne couvre que les lignes actives, plus petit et donc plus rapide à parcourir et à tenir à jour.

```
CREATE INDEX idx_commandes_client_actives
ON commandes(client_id)
WHERE deleted_at IS NULL;
```

Gain typique : un index environ deux fois plus léger, et un temps de requête qui passe d'une soixantaine de millisecondes à moins de dix.

Fonctions non-sargables : l'index existe, la requête l'empêche.

Ici, l'index est là, disponible. C'est la forme de la requête qui l'aveugle : une fonction, un OR mal posé, un joker en tête.

8. DATE() sur une colonne indexée PIÈGE CLASSIQUE

Requête lente : Un index existe sur **date_commande**, mais la requête enveloppe la colonne dans une fonction avant de comparer.

```
SELECT * FROM commandes
WHERE DATE(date_commande) = '2026-06-15';
```

Lecture du plan : Seq Scan on commandes Filter: (date(date_commande) = '2026-06-15'). Malgré l'index sur **date_commande**, le plan ignore complètement l'index disponible.

Goulot : un index btree classique est construit sur la valeur brute de la colonne, pas sur le résultat d'une fonction appliquée dessus; le prédicat n'est plus sargable, Postgres doit évaluer **DATE()** ligne par ligne.

Correction : réécrit en intervalle sargable sur la colonne brute, sans fonction; l'index redevient utilisable tel quel.

```
SELECT * FROM commandes
WHERE date_commande >= '2026-06-15'
AND date_commande < '2026-06-16';
```

Gain typique : d'un scan complet de quelques secondes à un accès index de quelques dizaines de millisecondes.

9. OR qui annule l'index disponible OR NON-SARGABLE

Requête lente : Un index existe sur **client_id**, mais une des branches du OR porte sur une colonne transformée par une fonction.

```
SELECT * FROM commandes
WHERE client_id = 48210
OR UPPER(email_contact) = 'TEST@MAIL.COM';
```

Lecture du plan : Seq Scan on commandes Filter: ((client_id = 48210) OR (upper(email_contact) = 'TEST@MAIL.COM')). Aucune des deux branches n'est traitée par index.

Goulot : dès qu'une branche d'un OR n'est pas sargable, Postgres doit évaluer tout le prédicat ligne par ligne : le bénéfice de l'index disponible sur l'autre branche (**client_id**) disparaît pour l'ensemble de la requête.

Correction : sépare les deux conditions en deux requêtes sargables combinées par UNION, chacune capable d'utiliser son propre index (et stocke l'email déjà normalisé en minuscule).

```
SELECT * FROM commandes WHERE client_id = 48210
UNION
SELECT * FROM commandes WHERE email_contact = 'test@mail.com';
```

Gain typique : de plusieurs secondes de scan complet à moins d'une centaine de millisecondes, une fois chaque branche indexable séparément.

10. LIKE avec un joker en tête NON SARGABLE

Requête lente : Recherche de clients par domaine d'adresse email sur une table **clients** de plusieurs millions de lignes, indexée en btree classique sur **email**.

```
SELECT nom, email FROM clients
WHERE email LIKE '%@entreprise-cible.com';
```

Lecture du plan : Seq Scan on clients Filter: (email LIKE '%@entreprise-cible.com'). Un index btree classique est inutile ici : il ne trie que par préfixe.

Goulot : un joker en tête de motif empêche tout index btree de restreindre la recherche à une plage triée : il n'existe aucun préfixe commun à exploiter.

Correction : deux options concrètes : stocker le domaine dans une colonne dédiée et indexée, ou passer par un index trigramme (**pg_trgm**) qui sait accélérer une recherche de sous-chaîne, y compris avec un joker en tête. Spécifique PostgreSQL : d'autres moteurs demandent un index full-text équivalent.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE INDEX idx_clients_email_trgm  
ON clients USING gin (email gin_trgm_ops);
```

Gain typique : d'un scan complet de plusieurs secondes à quelques dizaines de millisecondes via l'index trigramme.

Sargable ou pas : le test rapide

Un prédicat est sargable si Postgres peut l'évaluer directement sur la valeur indexée, sans la transformer d'abord.

Sargable

Colonne brute comparée directement : intervalle, égalité, préfixe.

Non sargable

Fonction ou joker en tête sur la colonne : DATE(), UPPER(), '%...'

Réflexe

Déplace la transformation sur la valeur cherchée, jamais sur la colonne.

Tri et agrégation : quand le calcul déborde.

Ici, la requête est correcte, mais son tri ou son regroupement demande plus de mémoire que ce qui est alloué.

11. Tri externe pour un ORDER BY sans index adapté **(SORT)**

Requête lente : Le dashboard des dernières commandes trie 5 millions de lignes sans aucun index sur la colonne de tri.

```
SELECT * FROM commandes  
ORDER BY date_commande DESC  
LIMIT 50;
```

Lecture du plan : Sort Method: external merge Disk: 184000kB, précédé d'un **Seq Scan**. Le tri porte sur la table entière avant d'appliquer le LIMIT.

Goulot : sans index sur **date_commande**, Postgres doit lire toute la table puis trier le résultat complet en mémoire, et déborde sur disque ("external merge") faute de place dans **work_mem**.

Correction : un index sur la colonne de tri permet de lire les lignes déjà triées et de s'arrêter dès que le LIMIT est atteint, sans jamais matérialiser le tri complet.

```
CREATE INDEX idx_commandes_date_desc  
ON commandes(date_commande DESC);
```

Gain typique : un tri externe de plusieurs secondes devient un parcours d'index de quelques millisecondes, le LIMIT arrêtant la lecture presque immédiatement.

12. GROUP BY qui déborde par mauvaise estimation **(HASHAGGREGATE)**

Requête lente : Un regroupement par client sur **commandes**, juste après un import massif qui a fait exploser la cardinalité réelle sans rafraîchir les statistiques.

```
SELECT client_id, SUM(montant) AS total
FROM commandes
GROUP BY client_id;
```

Lecture du plan : HashAggregate Disk Usage: 512000kB, alors que l'estimation prévoyait un nombre de groupes largement inférieur au réel.

Goulot : des statistiques périmées font sous-estimer le nombre de clients distincts, la table de hachage du regroupement est dimensionnée trop petite et déborde sur disque.

Correction : rafraîchir les statistiques en priorité; si le nombre de groupes reste réellement énorme, un GroupAggregate sur une entrée déjà triée par index peut éviter la table de hachage.

```
ANALYZE commandes;
```

Gain typique : avec des statistiques à jour et un plan correctement dimensionné, on passe d'un ordre de plusieurs secondes à moins d'une seconde.

Sous-requêtes : la bonne échelle d'exécution.

Ici, la requête est juste, mais son plan répète un calcul qui n'a besoin d'être fait qu'une fois.

13. Sous-requête corrélée exécutée à chaque ligne SUBPLAN RÉPÉTÉ

Requête lente : On veut la dernière commande de chaque client, avec une sous-requête scalaire dans le SELECT.

```
SELECT c.nom,
       (SELECT MAX(o.date_commande) FROM commandes o
        WHERE o.client_id = c.id) AS derniere_commande
FROM clients c;
```

Lecture du plan : SubPlan 1 -> Aggregate -> Index Scan, répété une fois par ligne de **clients**. Sur 200 000 clients, c'est 200 000 exécutions du même sous-plan.

Goulot : même avec un index sur **commandes.client_id**, une sous-requête corrélée dans le SELECT s'exécute ligne par ligne côté client externe : le coût grandit linéairement avec le nombre de clients.

Correction : remplace la sous-requête scalaire par une jointure sur un regroupement calculé une seule fois pour tous les clients.

```
SELECT c.nom, d.derniere_commande
FROM clients c
LEFT JOIN (
  SELECT client_id, MAX(date_commande) AS derniere_commande
  FROM commandes
  GROUP BY client_id
) d ON d.client_id = c.id;
```

Gain typique : d'un ordre de plusieurs secondes à quelques centaines de millisecondes, le calcul passant d'un par client à un seul global.

14. Sous-requête IN sans index côté table externe SEMI-JOIN

Requête lente : On veut les commandes des clients premium, en filtrant par une sous-requête IN plutôt qu'une jointure explicite.

```
SELECT * FROM commandes
WHERE client_id IN (
    SELECT id FROM clients WHERE segment = 'premium'
);
```

Lecture du plan : Hash Semi Join -> Seq Scan on commandes -> Hash (Seq Scan on clients Filter: segment = 'premium'). La sous-requête elle-même est bon marché, mais le côté **commandes** force un scan complet.

Goulot : le semi-join que Postgres construit pour le IN reste efficace pour trouver les clients premium, mais sans index sur **commandes.client_id**, il doit quand même lire toute la table **commandes** pour tester chaque ligne.

Correction : l'index à ajouter n'est pas sur la sous-requête, il est sur la table externe filtrée : c'est elle qui coûte le plus cher à scanner en entier.

```
CREATE INDEX idx_commandes_client_id
ON commandes(client_id);
```

Gain typique : d'un ordre de plusieurs secondes à quelques centaines de millisecondes, le semi-join pouvant alors s'appuyer sur un accès index côté commandes.

La méthode, en une checklist

Le réflexe transversal des 14 requêtes, réutilisable sur n'importe quelle requête lente.

Lire

EXPLAIN (ANALYZE, BUFFERS) avant de deviner.

Nommer

Seq Scan, Nested Loop, Sort externe, Batches > 1 : le goulot a un nom précis.

Vérifier

Un index n'aide que si le filtre est sélectif et le prédicat sargable.

Corriger

Réécriture d'abord, index ensuite, tuning mémoire en dernier recours.

Mesurer

Relance l'EXPLAIN après correction, ne te fie jamais à l'intuition.

Douter

Un plan qui s'exécute sans erreur n'est pas forcément un bon plan.

14 requêtes lues et corrigées. Tu n'as pas besoin de mémoriser chaque plan au mot près. Retiens le geste : lancer EXPLAIN avant de deviner, nommer le goulot avec précision, corriger la cause plutôt que d'ajouter un index au hasard. C'est ce réflexe qui distingue un analyste qui écrit du SQL de quelqu'un à qui on peut confier une base en production.

Fait main, en solo.

Si ce guide t'a aidé à débloquer une requête lente, partage-le autour de toi : il peut débloquer un autre analyste. Et si tu veux que je traite un plan précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

